

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]

14

Query Planning & Optimization – Part I



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Mid-Term Exam is Wed Oct 16th @ 12:00pm
→ See [mid-term exam guide](#) for more info.

Project #2 is due Sun Oct 20th @ 11:59pm

QUERY OPTIMIZATION

Remember that SQL is declarative.

→ User tells the DBMS what answer they want, not how to get the answer.

There can be a big difference in performance based on plan is used:

→ See last week: 1.3 hours vs. 0.45 seconds

IBM SYSTEM R

First implementation of a query optimizer from the 1970s.

→ People argued that the DBMS could never choose a query plan better than what a human could write.

Many concepts and design decisions from the **System R** optimizer are still used today.

QUERY OPTIMIZATION

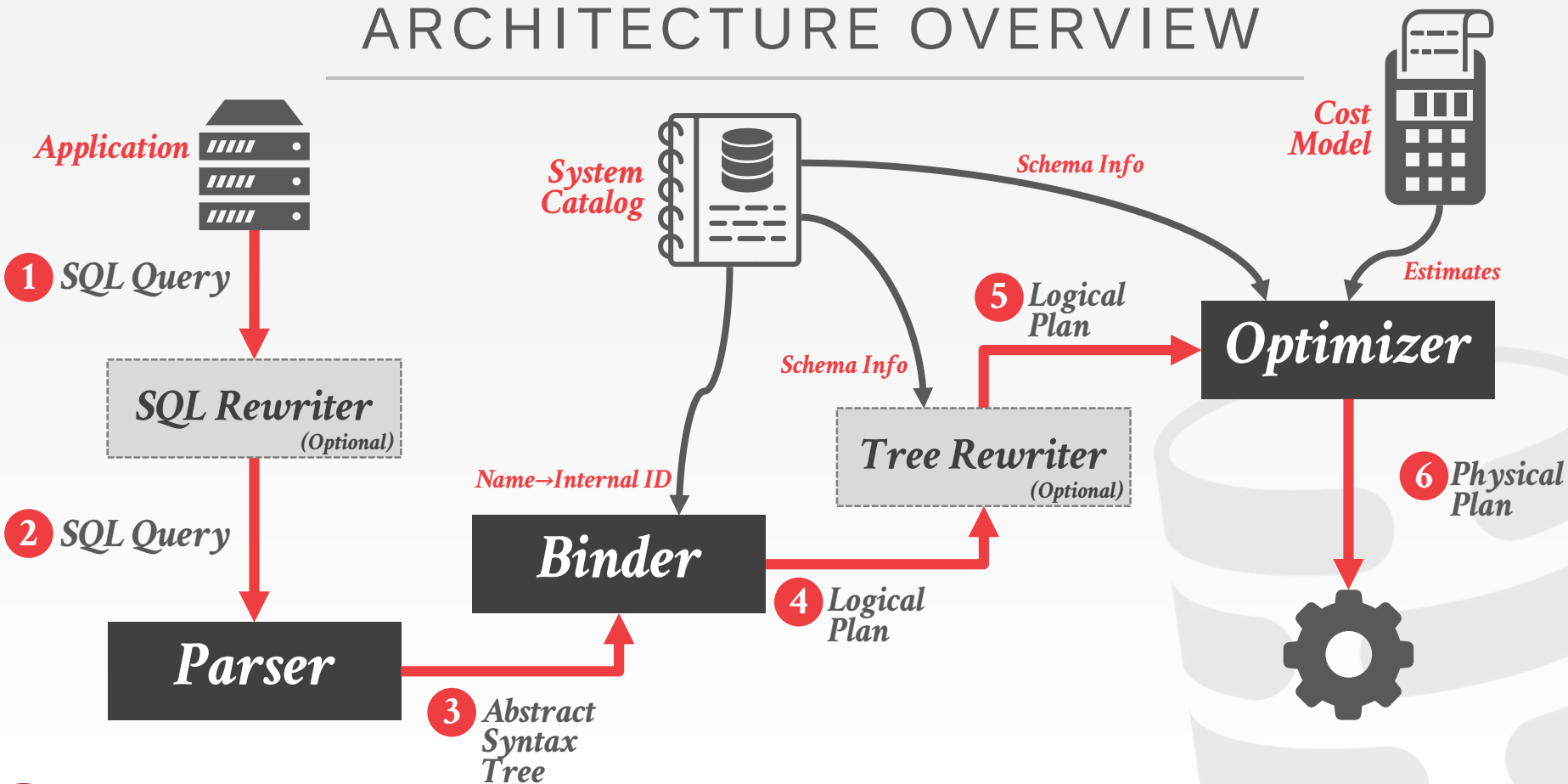
Heuristics / Rules

- Rewrite the query to remove stupid / inefficient things.
- These techniques may need to examine catalog, but they do not need to examine data.

Cost-based Search

- Use a model to estimate the cost of executing a plan.
- Evaluate multiple equivalent plans for a query and pick the one with the lowest cost.

ARCHITECTURE OVERVIEW



LOGICAL VS. PHYSICAL PLANS

The optimizer generates a mapping of a logical algebra expression to the optimal equivalent physical algebra expression.

Physical operators define a specific execution strategy using an access path.

- They can depend on the physical format of the data that they process (i.e., sorting, compression).
- Not always a 1:1 mapping from logical to physical.

QUERY OPTIMIZATION IS NP-HARD

This is the hardest part of building a DBMS.

If you are good at this, you will get paid \$\$\$.

People are starting to look at employing ML to improve the accuracy and efficacy of optimizers.

I am expanding the Advanced DB Systems class to cover this topic in greater detail.

TODAY'S AGENDA

Relational Algebra Equivalences

Static Rules



RELATIONAL ALGEBRA EQUIVALENCES

Two relational algebra expressions are equivalent if they generate the same set of tuples.

The DBMS can identify better query plans without a cost model.

This is often called **query rewriting**.

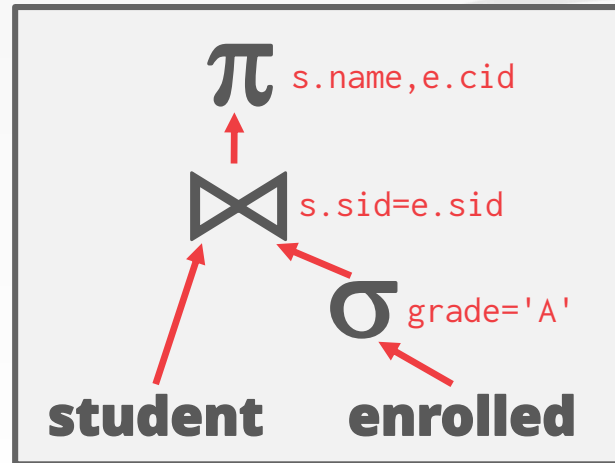
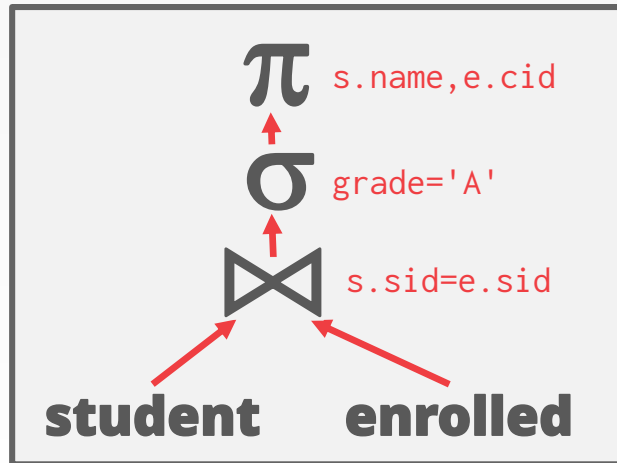
PREDICATE PUSHDOWN

```
SELECT s.name, e.cid  
  FROM student AS s, enrolled AS e  
 WHERE s.sid = e.sid  
    AND e.grade = 'A'
```

$\pi_{\text{name, cid}}(\sigma_{\text{grade='A'}}(\text{student} \bowtie \text{enrolled}))$

PREDICATE PUSHDOWN

```
SELECT s.name, e.cid  
FROM student AS s, enrolled AS e  
WHERE s.sid = e.sid  
AND e.grade = 'A'
```



RELATIONAL ALGEBRA EQUIVALENCES

```
SELECT s.name, e.cid
FROM student AS s, enrolled AS e
WHERE s.sid = e.sid
AND e.grade = 'A'
```

$$\pi_{\text{name, cid}}(\sigma_{\text{grade='A'}}(\text{student} \bowtie \text{enrolled}))$$
$$=$$
$$\pi_{\text{name, cid}}(\text{student} \bowtie (\sigma_{\text{grade='A'}}(\text{enrolled})))$$

RELATIONAL ALGEBRA EQUIVALENCES

Selections:

- Perform filters as early as possible.
- Reorder predicates so that the DBMS applies the most selective one first.
- Break a complex predicate, and push down

$$\sigma_{p1 \wedge p2 \wedge \dots \wedge pn}(R) = \sigma_{p1}(\sigma_{p2}(\dots \sigma_{pn}(R)))$$

Simplify a complex predicate

$$\rightarrow (X=Y \text{ AND } Y=3) \rightarrow X=3 \text{ AND } Y=3$$

RELATIONAL ALGEBRA EQUIVALENCES

Projections:

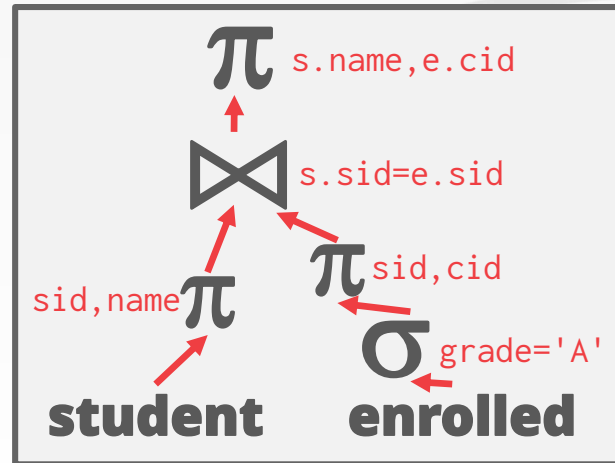
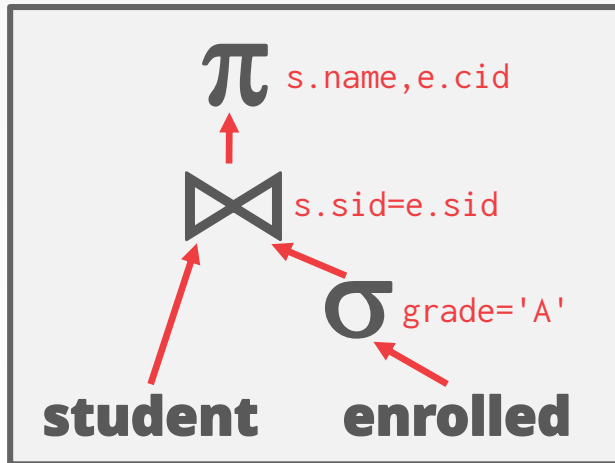
- Perform them early to create smaller tuples and reduce intermediate results (if duplicates are eliminated)
- Project out all attributes except the ones requested or required (e.g., joining keys)

This is not important for a column store...

PROJECTION PUSHDOWN

```

SELECT s.name, e.cid
FROM student AS s, enrolled AS e
WHERE s.sid = e.sid
AND e.grade = 'A'
  
```



```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0; X
```

Source: [Lukas Eder](#)

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0; X
```

```
SELECT * FROM A WHERE 1 = 1;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0; X
```

```
SELECT * FROM A;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0; X
```

```
SELECT * FROM A;
```

Join Elimination

```
SELECT A1.*  
  FROM A AS A1 JOIN A AS A2  
    ON A1.id = A2.id;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Impossible / Unnecessary Predicates

```
SELECT * FROM A WHERE 1 = 0; X
```

```
SELECT * FROM A;
```

Join Elimination

```
SELECT * FROM A;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Ignoring Projections

```
SELECT * FROM A AS A1  
WHERE EXISTS(SELECT val FROM A AS A2  
              WHERE A1.id = A2.id);
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Ignoring Projections

```
SELECT * FROM A;
```



```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Ignoring Projections

```
SELECT * FROM A;
```

Merging Predicates

```
SELECT * FROM A  
  WHERE val BETWEEN 1 AND 100  
         OR val BETWEEN 50 AND 150;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Ignoring Projections

```
SELECT * FROM A;
```

Merging Predicates

```
SELECT * FROM A  
WHERE val BETWEEN 1 AND 100  
OR val BETWEEN 50 AND 150;
```

```
CREATE TABLE A (  
  id INT PRIMARY KEY,  
  val INT NOT NULL );
```

MORE EXAMPLES

Ignoring Projections

```
SELECT * FROM A;
```

Merging Predicates

```
SELECT * FROM A  
WHERE val BETWEEN 1 AND 150;
```

RELATIONAL ALGEBRA EQUIVALENCES

Joins:

→ Commutative, associative

$$R \bowtie S = S \bowtie R$$

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

How many different orderings are there for an n -way join?

RELATIONAL ALGEBRA EQUIVALENCES

How many different orderings are there for an n -way join?

Catalan number $\approx 4^n$

→ Exhaustive enumeration will be too slow.

We'll see in a second how an optimizer limits the search space...

CONCLUSION

We can use static rules and heuristics to optimize a query plan without needing to understand the contents of the database.

NEXT CLASS

MID-TERM EXAM!

→ Seriously, this is not a joke.



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]